

ERRORES CLÁSICOS EN LA GESTIÓN DE PROYECTOS TI¹

Si bien disponer de un sólido conocimiento de lo que no se debe hacer en un proyecto supone un logro importante, hay que insistir en que no es suficiente con identificar los principales riesgos, que es lo que hace este documento a grandes rasgos, hay que afrontarlos de forma proactiva. Es decir, durante el desarrollo del proyecto hay que seguir monitorizándolos, analizarlos, priorizarlos y resolverlos. Muchos de los puntos aquí listados parecerán evidentes, pero en la realidad se dan desgraciadamente con elevada frecuencia. En la mayoría de los casos esto se debe a la subestimación de la magnitud de su impacto sobre el proyecto y factores psicológicos que hacen que las decisiones se toman más por lo que "pide el cuerpo" en un momento determinado que por razones objetivas. Para ver las magnitudes de las que estamos hablando, he aquí un ejemplo real:

UN EJEMPLO: MICROSOFT WORD 1.0₁

El ejemplo de Microsoft Word 1.0 supone una buena lección en lo que refiere a las consecuencias de una mala planificación. Esta aplicación costó 5 años de desarrollo, consumiendo 660 hombres/mes² en esfuerzo y produjo un sistema de 249.000 líneas de código. WinWord tuvo una planificación extremadamente agresiva, la planificación del mejor caso para un proyecto de esta envergadura es de unos 460 días, la estimación para éste del caso más largo admitida era de 395 días, una diferencia de 65 días.

La productividad ha sido finalmente de 377 líneas de código por hombre/mes cuando una media de 1000-1500 líneas de buena calidad suele ser habitual. Esto da una idea del nivel de degradación que sufrió el proyecto.

¿Cómo fue posible tal desmesura? El desarrollo de WinWord cumplió con una serie de ejemplos típicos de cosas que pueden ir mal cuando un proyecto software se planifica demasiado agresivo:

- Winword sufría objetivos inalcanzables. La directiva de Bill Gates para el equipo fue "hacer el mejor procesador de texto de todos los tiempos" y hacerlo lo más rápido posible, preferiblemente en 12 meses, cualquiera de los dos objetivos era un desafío, los dos juntos eran sencillamente imposibles.
- Los tiempos agresivos impidieron una planificación precisa. En los cinco años se hicieron un total de 17 estimaciones de las cuales sola la segunda supero el año (se hizo nueve meses después de comenzar).
- El proyecto sufrió una rotación de personal extrema, tuvo 4 líderes distintos a lo largo de los cinco años, dos que abandonaron el proyecto por presión de tiempo excesiva y uno por razones de salud.
- Con la presión los desarrolladores descuidaron la calidad incluso de las partes críticas de la aplicación, diciendo que estaban "hechas", aun a sabiendas que no tenían la calidad necesaria y estaban incompletas. El resultado fueron 12 meses de "estabilización", cuando este período se estimaba en 3 meses.

El tiempo de desarrollo nominal (es decir, holgado) que se estimó después de un análisis a posteriori eran 26 meses, un tiempo de desarrollo "eficiente", 22 meses. Este tiempo hubiera sido sustancialmente más largo que el tiempo más pesimista planificado, pero hubiese dado el margen suficiente para evitar el grado de presión que degradó el proyecto por completo. Es doloroso asumir un plazo de 22 meses para un proyecto que se quiere obtener en 12, pero forzar el plazo deseado no es la solución como este ejemplo demuestra contundentemente. **El plazo más corto es el que con mayor acierto se ha estimado.** Ejemplos de compañías que han aprendido estas lecciones incluyen Boeing, la NASA, Microsoft, Ericsson, etc. Ellos utilizan planificaciones

¹ Extracto de "Cómo Gestionar Proyectos IT con Éxito", Advanced Quality Solutions, 2002.

sistemáticas y realistas para evitar estos desastres y asumen los tiempos que los técnicos indican de cara a obtener un resultado determinado.

Ahora estamos en buenas condiciones para comprender la importancia de los siguientes párrafos:

ERRORES RELACIONADOS CON EL FACTOR HUMANO

- **Motivación insuficiente.** Estudio tras estudio se ha demostrado que el factor que más afecta a la productividad y calidad del desarrollo probablemente sea la motivación del personal. Resulta fácil degradarla con presiones por calendarios excesivos, y decisiones irracionales y por tanto hay que saber gestionarla. Este es un punto tan importante que le dedicamos un capítulo aparte, pero adelantemos aquí ya algunas ideas. Hay tres factores principales en la motivación de los desarrolladores:
 - Las relaciones humanas y personal adecuado en los equipos. El personal de la empresa no solamente se debe seleccionar por criterios puramente técnicos, sino también humanos. Resulta tan fundamental que los jefes de proyecto sean competentes en el ejercicio de su función como que tengan una personalidad adecuada. Por otra parte resulta muy recomendable buscar no solamente la cualificación técnica en los desarrolladores, sino intentar contratar también a “buena gente”, un equipo que se lleva bien entre sí siempre será mucho más productivo que un equipo con un ambiente frío o incluso enemistado. El buen estar entre las personas del equipo conllevará casi como efecto secundario una alta productividad. Es fácil caer en la trampa de contratar a gente nueva por el plazo de su disponibilidad en vez de contratarla por la aportación al proyecto durante su tiempo de vida, hay que evitar este error a toda costa.
 - La relación entre la empresa y su personal. Decisiones aparentemente irracionales, promesas incumplidas y falta de comunicación son posiblemente los factores que más enturbian la relación de los empleados con la empresa. La empresa debe cuidar la comunicación con sus empleados y procurar que adquieran un poco de cultura de empresa. Si el equipo directivo logra de esta manera la confianza del personal se respetarán incluso aquellas decisiones que puedan parecer “irracionales”, para ello no es necesario ni operativo explicar cada una de las decisiones estratégicas que toma la empresa, pero sí resulta recomendable escoger ocasiones puntuales para hacerlo, mantener sesiones periódicas de información sobre el estado de la empresa y conseguir que los desarrolladores se sensibilicen ante los problemas de gestión y negocio de la empresa. Cuanto más se sientan implicados los desarrolladores con la empresa, más se implicarán con sus proyectos.
 - La filosofía de trabajo. Varias estadísticas demuestran que la mayoría de los desarrolladores con cierta vocación ponen por encima de criterios económicos lo que les aporta su trabajo, hay dos facetas principales: A) Una forma de trabajo ordenada y participativa en la que cada uno de los miembros se sienta partícipe. B) El desarrollo personal de cada trabajador. Una gestión descontrolada del trabajo supone objetivos cambiantes constantes, estrés, falta de sensación de éxito y con ello frustración. Deteriora la moral de las personas y por tanto la calidad de su trabajo culminando en que tarde o temprano la persona se irá de la empresa. Esta situación se produce por una combinación de factores compleja y que se examinan uno a uno en los siguientes puntos del presente documento, limitémonos por ahora a decir, que una forma de trabajo desordenada es sencillamente, inadmisible. En cuanto al desarrollo personal resulta importante establecer una gestión en la que se den facilidades a cada

empleado. Aquí no solamente cuentan las medidas costosas al alcance de las grandes compañías como planes de formación, etc. sino que pueden ser incluso mucho más eficaces medidas más "creativas" como "institucionalizar" simplemente un tiempo diario de investigación para cada desarrollador, la organización de eventos de intercambio de know-how entre los proyectos (workshops internos), animar a que se redacten artículos específicos sobre experiencias concretas en proyectos o nuevas tecnologías, tanto a nivel de gestión como tecnológico, serán una valiosa aportación a la gestión de conocimiento de la empresa y además se puede plantear la posibilidad de publicarlos, etc.

- **Empleados conflictivos.** No gestionar los problemas relacionados con personal problemático provoca también retrasos en el desarrollo de un proyecto. Es un problema común y se ha entendido bien desde que Gerald Weinberg publicó *Psychology of Computer Programming* en 1971. No actuar ante una situación así es la queja más frecuente de los miembros de un equipo sobre sus líderes (Larson y LaFasto 1989). La solución no tiene que pasar necesariamente por medidas radicales, sino que se debe gestionar con cierta delicadeza, muchas veces se debe a una determinada situación personal, falta de experiencia que lleva a malinterpretar la forma de actuar de su entorno, etc. Si se consiguen explorar las razones de fondo del comportamiento de la persona en cuestión muchas veces hay una solución que resultará positiva tanto para la(s) persona(s) como para la empresa y que incluso puede fortalecer los vínculos entre ambos.
- **Heroísmos.** Algunos desarrolladores piensan que acciones heroicas pueden aportar alguna clase de beneficio al proyecto (Bach 1995). Pero las experiencias demuestran que más bien son actitudes negativas para la gestión fiable de un proyecto puesto que problemas potenciales no se comunican hasta el último minuto distorsionando así el seguimiento del proyecto y se tiende a asumir riesgos demasiado altos. Este error es también muy seductor porque para muchos jefes de proyecto resulta tranquilizador que su equipo les transmita mensajes del tipo "lo podemos hacer", el jefe de proyecto se limita a creerlo y dormir más tranquilo, pero realmente está obviando la realidad y aplazando enfrentarse a una problemática que se agravará con el tiempo.
- **Añadir personal a un proyecto retrasado.** Este es, sin duda, el fallo clásico más clásico. Y es que el factor humano no escala fácilmente, generalmente añadir gente a un equipo existente que se encuentra retrasado lo retrasa aún más puesto que genera mucho trabajo de coordinación adicional, además el equipo no cuenta en ese momento con la calma necesaria para dar a las personas nuevas la formación que necesitan del proyecto, con lo cual ésta irá lenta. Al final, la combinación de más canales de comunicación y necesidad de formación hace que los nuevos miembros del equipo le restan más tiempo de productividad al proyecto del que añaden y generan así retrasos aún mayores. Aunque de todas las maneras hay que hacer notar que esto depende evidentemente del perfil de cada proyecto, hay proyectos en los cuales sí resulta eficaz introducir más personal ante retrasos por requerir éste solamente una forma mínima o poder tener una estructura de módulos muy independientes. Lo que resulta importante es comprender es que añadir más gente a un proyecto no es ni mucho menos una fórmula mágica para resolver problemas de calendario, sino que hay una muy alta probabilidad de que sea todo lo contrario.
- **Oficinas ruidosas y faltas de espacio.** La mayoría de los desarrolladores valoran sus condiciones de trabajo como insatisfactorias, aproximadamente el 60% se queja además de que no hay suficiente silencio ni suficiente intimidad (DeMarco y Lister 1987). Trabajadores que desarrollan su actividad en oficinas silenciosas y con espacio tienden a ser más notablemente más eficientes que aquellos que trabajan en condiciones de ruido y falta de espacio.

- **Fricción entre desarrolladores y clientes.** Fricciones entre desarrolladores y clientes pueden surgir de varias maneras. Los clientes opinan que los desarrolladores no cooperan cuando rechazan el calendario exigido por ellos o cuando no entregan en los tiempos prometidos. Los desarrolladores piensan que los clientes insisten irracionalmente en calendarios irrealistas y cambios de requisitos imposibles después de que éstos hayan sido cerrados, las personalidades de ambos grupos pueden empeorar el conflicto aún más. La consecuencia principal es una comunicación pobre y por tanto una comprensión pobre de los requerimientos, junto con interfaces de usuario poco adecuados a las necesidades del cliente. Resulta difícil superar esta fricción por lo que hay que monitorizar este riesgo desde el principio detenidamente y emplear personal con “mano izquierda” que sepa resolver los conflictos y restaurar una comunicación eficaz.
- **Expectativas irrealistas.** Una de las causas más frecuentes de fricción entre desarrolladores y clientes o gestores son expectativas irrealistas. Sensibilizar al cliente no es un proceso trivial y requiere también una postura suficientemente razonable de su parte. Una vía de compromiso ante plazos que al cliente le parecen excesivos puede ser una cuidadosa negociación de los requisitos. Si se sabe negociar bien se encontrarán en la mayor parte de los casos requisitos que para el cliente resultan relativamente poco importantes pero sí suponen un ahorro de desarrollo que merece la pena. Otra vía puede ser una planificación orientada a prototipos o entregas sucesivas de módulos que adelanten funcionalidad importante aunque la versión completa se entregue en los plazos iniciales o incluso algo más tarde. Con este tipo de medidas posiblemente se puedan acercar posiciones en un principio alejadas.
- **Ausencia de apoyo efectivo de la dirección.** Todos los principios para la gestión exitosa de proyectos valen de poco si no son apoyados por la dirección de la empresa y ésta se deja llevar excesivamente por la presión de sus clientes forzando a los equipos a aceptar planificaciones irrealistas, etc. Según el consultor australiano Rob Thomsett esta falta de apoyo prácticamente garantiza el fracaso de un proyecto con cierto grado de ambición (Thomsett 1995).
- **Compromiso insuficiente de los participantes en el proyecto.** Todos los participantes en un esfuerzo de desarrollo de software tienen que estar fuertemente comprometidos con el proyecto, tanto del lado del cliente como del lado de la empresa desarrolladora. Compromiso quiere decir fundamentalmente dedicar el tiempo y la profundidad necesaria a las tareas que correspondan a cada uno. Esto incluye tanto a los ejecutivos, como miembros del equipo de desarrollo, interlocutores del cliente, usuarios finales y cualquiera que se encuentre relacionado con el proyecto. Solamente de esta manera será posible una buena coordinación y capacidad de maniobra ante problemas.
- **Falta del “input” de usuario.** Resulta muy difícil para los no informáticos imaginar hasta qué punto se necesita el detalle de los procesos para implementarlos bien. TODOS los clientes tienden a quedarse muy en la superficie de los requerimientos, si los responsables del desarrollo lo admiten, esto se traduce normalmente en desarrollos que están lejos de cubrir las necesidades del cliente y con ello en retrasos para lograrlo, lo que a su vez provoca código “sucio” por los frecuentes cambios, mucho más difícil de mantener y evolucionar. Por otra parte resulta difícil para los desarrolladores imaginar hasta qué punto los usuarios se quedan en la superficie, ya que muchos requisitos que para ellos resultan evidentes los usuarios ni son capaces de imaginárselos y viceversa². Debe tenerse en cuenta este dato: la Standish Group

² Steve McConnell pone un ejemplo muy ilustrativo de la distancia que separa los puntos de vista de usuarios y desarrolladores: propone que el lector se imagine un cliente experto en coches (pero no en ingeniería) que debe especificar la construcción de un coche a medida a un ingeniero que no es un especialista en la construcción de coches porque construye igualmente aviones, barcos, etc. El cliente hablará del motor, chasis, lunas, volante, etc. Pero imaginemos que se le olvidó especificar que al dar marcha atrás se deben encender unas luces blancas en la parte trasera del coche (algo evidente para él).

concluyó que la razón principal por la que los proyectos de software concluyen con éxito es por la participación comprometida de los usuarios.

- **Demasiados interlocutores o interlocutores ineficaces.** Normalmente se da este problema más de parte del cliente debido a que la participación en el proyecto para él supone una actividad con la que debe compaginar su trabajo diario, normalmente los responsables del lado del cliente prometen que las personas designadas se podrán alejar de su responsabilidad habitual para centrarse en el proyecto, pero en la realidad pocas veces se cumple, al final tienen que abordar todo su trabajo habitual más el trabajo del proyecto. El proyecto muchas veces se convierte en la segunda prioridad porque ante su empresa se les valora por su trabajo habitual, esto sencillamente resulta nefasto para cualquier proyecto. Otra faceta en la comunicación es el número de interlocutores, debería ser mínimo, con áreas de responsabilidad muy bien definidas y acotadas y con una dedicación plena. Aunque esto es difícil de conseguir resulta muy recomendable presionar al máximo al cliente para acercarse lo más posible a este ideal, el cliente lo agradecerá cuando el proyecto se haya realizado con éxito.
- **Intereses “políticos”.** Este punto se refiere tanto a conflictos de intereses entre los participantes en el proyecto, típicamente departamentos o responsables con malas relaciones, como miembros del proyectos cuyas pautas de actuación derivan más de la imagen hacia sus jefes y situación coyuntural que los intereses del proyecto o la empresa. Esta problemática resulta habitualmente invisible para los cargos más altos, debida a su falta general de tiempo que les hace recibir poca información la cual además fácilmente desvirtúa la realidad (o puede ser desvirtuada intencionadamente). Sin embargo, es interesante observar que la tasa de éxito de las estrategias “tropa” es considerablemente inferior de lo que comúnmente se piensa (Constantine 1995a).
- **“Wishful thinking”.** Esto es básicamente el fenómeno de ponerse la venda en los ojos ante la voluntad que querer cumplir de cualquier manera un determinado objetivo por imposible que sea, ipero no hay que confundirlo con optimismo, el optimismo de por sí es sano para un proyecto! Cuantas veces se habrán escuchado frases como estas:
 - “Ninguno de los miembros del equipo realmente aceptada los plazos del proyecto, pero pensaron que trabajando duro y si las cosas salían bien, podrían ser capaces de sacarlo adelante.”
 - “No ha hecho falta coordinar mucho los interfaces entre los módulos del producto, hemos estado hablando bastante y son relativamente simples, así que no debería haber muchos problemas en la integración.”
 - “Al final hemos tenido que subcontratar un equipo de menor cualificación de lo especificado para el módulo de acceso a base de datos, pero son gente joven y muy motivados, seguro que lo sacan adelante a tiempo.”
 - “No hace falta enseñar al cliente los últimos cambios al prototipo, ya le conocemos bien y le encantará como haremos la versión final.”
 - “El equipo dice que va hacer un esfuerzo extraordinario para cumplir el plazo, se retrasaron en el primer hito el otro día, pero yo creo que a eso no hay que darle mucha importancia.”
 - ...

El ingeniero va hacer su trabajo y vuelve a los seis meses, cuando el experto ve el coche exclama “Vaya, me he olvidado especificar las luces de marcha atrás”. El ingeniero se lleva las manos a la cabeza: “¿sabe usted lo que va a costar hacer esta modificación? ¡Tenemos que rediseñar la parte posterior del coche para incluir los faros nuevos, modificar la circuitería electrónica e incluir un sensor en el cambio! Esto llevará semanas como mínimo. ¿Porqué no me lo ha dicho antes?” El experto se resigna: “Parecía una petición tan sencilla...”

Este fenómeno es muy frecuente y extremadamente peligroso, resulta frecuente tanto del lado del cliente que tiende a ignorar las valoraciones de los técnicos pensando que ya habrá una manera de hacer lo que necesita en el plazo que necesita como del lado de los desarrolladores que se ponen a trabajar duro para alcanzar “como sea” la fecha límite sin plantearse si es posible.

- **Confianza ciega en referencias establecidas.** Esto es básicamente el fenómeno de confiar hasta tal punto en grandes nombres que no se contrasta la cualificación que ofrecen. Es decir, contratar el equipo de una gran marca por si solo no implica en absoluto una garantía de calidad, hay que contrastar siempre la calidad del equipo que aportan.

ERRORES RELACIONADOS CON EL PROCESO

- **Planificación excesivamente optimista.** Una planificación excesivamente optimista impide una planificación eficiente y genera presiones que acaban degradando al equipo y proyecto en la mayoría de los casos. Una faceta posiblemente peor aún de una planificación excesivamente optimista es que no solamente no se llega a tiempo a los objetivos, sino que la dinámica en la que entra un proyecto en estas condiciones por la presión del tiempo hace que la productividad del trabajo sea sensiblemente menor que en un proyecto correctamente planificado.
- **Gestión de riesgos insuficiente.** Resulta muy fácil llegar a un punto en el cual no queda capacidad de maniobra si se detecta que el proyecto va mal cuando no hay una gestión de riesgos activa. Este hecho es especialmente relevante si se tiene en cuenta que muchas veces un solo factor de los aquí expuestos puede degradar el proyecto por completo.
- **Fallos debidos a la subcontratación.** A veces se subcontratan partes de un proyecto que por presiones del calendario y falta de personal especializado. Pero las empresas subcontratadas con frecuencia entregan el trabajo tarde, con un nivel de calidad inaceptable o no conforme a los requisitos (Boehm 1989).
- **Abandonar la planificación ante la presión del tiempo.** El problema no es tanto el abandono de la planificación en sí como el hecho de que casi todos los equipos de desarrollo caen en la trampa psicológica de entrar en el “code-and-fix” (programar y arreglar a toda velocidad sin demasiada meditación) ante presiones de tiempo altas. El trabajo a partir de ese punto se convierte en descoordinado y más ineficiente que antes.
- **Pérdida de tiempo en la fase preparativa de un proyecto.** La fase preparativa de un proyecto es el tiempo que transcurre entre su aprobación y su presupuestado. No es raro que este proceso dure meses, a veces incluso años y luego plantee, sin embargo, una planificación de la ejecución del proyecto muy agresiva. Es mucho menos arriesgado acortar la fase preparativa y disponer de tiempo para la ejecución.
- **Plantear demasiados objetivos a la vez.** Este error es otro de los “muy frecuentes”, tanto a nivel microscópico como macroscópico. El punto de vista microscópico se refiere al día a día de la gestión del proyecto, es decir, los objetivos a corto plazo que el jefe de proyecto va planteando al equipo. Plantear demasiados objetivos en paralelo es uno de los factores que en mayor medida impiden que un equipo trabaje de forma efectiva. El punto de vista macroscópico se refiere a los objetivos que se plantean para el producto, véase el punto “Plantear demasiados objetivos a la vez” de la sección de errores relacionados con el producto.

- **Cortar actividades fundamentales ante un retraso.** En muchos proyectos acciones de recorte de actividades han sido, por ejemplo, no dedicar el tiempo necesario a la especificación de requisitos, análisis y diseño. Recortar actividades fundamentales como éstas desde luego empeora con garantías aún más la situación.
- **Diseño inadecuado.** Este es un caso particular del punto anterior, pero especialmente delicado por el alto grado de creatividad y por tanto de cierta calma que requiere para ser realizado con éxito. Muchas veces la presión convierte las decisiones de diseño en decisiones oportunistas frente a las necesidades del momento que provocan la necesidad de uno o más rediseños del sistema antes de entregar el proyecto. Finalmente los rediseños suman más tiempo del que hubiese requerido un diseño bueno desde el principio.
- **Control de calidad insuficiente.** Otro punto típico de recorte es el control de calidad, es decir, eliminar actividades como la revisión del diseño y el código, planes de pruebas, etc. Estudios demuestran que recortar un día de actividades QA (Quality Assurance) en las fases tempranas de un proyecto es probable que cueste entre 3 a 10 días de actividad posterior (Jones1994). Es decir, al final se alarga el proyecto.
- **Control de gestión insuficiente.** Es importante llevar a cabo un control de la evolución del desarrollo para ver si los hitos planteados se van cumpliendo y diagnosticar las causas de los posibles problemas puesto que esto permite reaccionar a tiempo y tomar decisiones razonables dentro de la situación del proyecto.
- **Convergencia prematura.** Poco antes del momento planificado para la entrega de un producto, hay una fase frenética para pulir la velocidad, la documentación final, incorporar ayuda en línea, rematar el programa de instalación, etc. En proyectos bajo presión existe la tendencia de entrar en esta fase demasiado pronto, lo que repercute en repetirla media docena o más de veces hasta que realmente se concluye.
- **Omitir tareas necesarias de las estimaciones.** En todos los proyectos hay un alto porcentaje de tareas y requisitos poco evidentes, pero necesarios. Aquí entran puntos como tiempos de respuesta del sistema, determinados aspectos ergonómicos del interfaz de usuario, procesos secundarios (como backups) derivados de los procesos especificados, etc. Su omisión se penaliza frecuentemente con desviaciones entre un 20% y un 30% sobre el tiempo total del proyecto.
- **Planificar para recuperar retrasos dentro de la planificación inicial.** Una manera muy frecuente de corregir estimaciones ante un retraso frente a un hito determinado es planificar una compensación posterior sin alterar los tiempos globales del proyecto. Es decir, cuando sobre un proyecto de 6 meses se llega al hito del mes 2 en el mes 3 es habitual decir "ya lo recuperaremos en lo que queda de proyecto, tenemos aún 3 meses por delante y ya sabemos más del proyecto". Aunque es cierto que la productividad de por sí crece conforme se conoce mejor al producto desarrollado, la mayoría de los proyectos no recuperan nunca de esta manera sus retrasos iniciales.
- **Programación "code-like-hell".** Algunas compañías creen que una codificación rápida con gente motivada es lo que hace falta realmente para finalizar los proyectos en el menor tiempo y que de esa manera se supera cualquier obstáculo. Los riesgos expuestos en este capítulo y sus razonamientos deberían dejar claro que están muy equivocados. Recientemente han aparecido propuestas como el "Extreme Programming (XP)" que no se deben confundir con en esta filosofía.

ERRORES RELACIONADOS CON EL PRODUCTO

- **Exceso en los requerimientos.** Muchos proyectos tienden a incluir más requerimientos de los que realmente necesitan, requisitos complejos añaden tiempos de desarrollo desproporcionados con respecto a los requisitos fundamentales. Luego hay analizar muy bien cuales quiere realmente el cliente para su negocio y priorizarlos correctamente.
- **Plantear demasiados objetivos a la vez.** Relacionado con el producto se refiere al nivel macroscópico, es decir, a los objetivos globales del proyecto del proyecto. Pedir que un proyecto justo en plazos sea además óptimo en el uso de la memoria, óptimo en velocidad de respuesta, óptimo en facilidad de uso y tenga una legibilidad del código perfecta será un conjunto de objetivos prácticamente imposible de alcanzar. Asumir un consumo relativamente elevado de memoria (es un recurso muy barato hoy en día) a cambio de una máxima velocidad de respuesta, junto con una facilidad de uso buena (si el perfil de usuarios lo permite) y una legibilidad de código aceptable puede ser una sabia rectificación en virtud del éxito del proyecto y producirá una calidad del resultado prácticamente similar.
- **Requerimientos inestables.** El enemigo más feroz de la calidad del producto es el constante cambio de los requerimientos puesto que hacen sencillamente imposible un buen diseño, de hecho es uno de los factores principales en el fracaso de los proyectos. Es muy importante concienciar al usuario de esto, ha de tenerse en cuenta que la informática es una ingeniería de tecnología puntera, con un alto grado de dificultad que requiere diseños cuidadosos.
- **Desarrollo no centrado en los objetivos del proyecto por parte de miembros del equipo de desarrollo.** Muchos programadores tienden a fascinarse con los nuevos avances tecnológicos y con la implementación de “virguerías”, tanto por disfrute personal como por la ambición de superar ciertos retos. Esta tendencia tiene que ser controlada porque puede degenerar en una pérdida de tiempo importante, aunque también se debe respetar algo de libertad para mantener la motivación a un buen nivel. Aquí el criterio personal del jefe de proyecto es la clave.
- **Negociación “tira y afloja”.** Un tipo de negociación estrafalaria sucede cuando un responsable que aprueba un retraso en la planificación de un proyecto que transcurre más lento de lo esperado añade a la vez tareas completamente nuevas después de la aprobación de los nuevos plazos. Muchas veces hay motivos psicológicos de presión ante superiores que hacen que el responsable se sienta forzado a “compensar” el retraso con más funcionalidad (muchas veces poco útil, además), pero este alivio a corto plazo es traidor porque realmente contribuirá a alargar el proyecto aún más y el enfrentamiento con sus superiores será peor.
- **Desarrollo orientado a investigación.** El estar en la cresta de la tecnología convierte la planificación en pura especulación, hay que encontrar el punto intermedio óptimo entre los beneficios de las nuevas tecnologías y las áreas de aplicación de éstas en el desarrollo. En este sentido un apoyo I+D externo al proyecto es de gran utilidad, ya que con su ayuda se podrá calibrar la fiabilidad y potencial de mejora de la nueva tecnología y decidir así con criterio si la relación entre beneficio y riesgo hace recomendable utilizarla o no en un determinado proyecto.

ERRORES RELACIONADOS CON LA TECNOLOGÍA

- **El síndrome de la panacea.** La creencia ciega en tecnología y herramientas que no han sido probadas y se consideran la solución mágica a todos los problemas, la tendencia a esto es elevada en muchas personas.
- **Ahorros sobreestimadas por herramientas o métodos nuevos.** Los beneficios de nuevas herramientas o métodos de trabajo se ven anulados a corto/medio plazo por la curva de aprendizaje que supone su dominio correcto. Eso hace difícil que dentro de un proyecto se consigan resultados espectaculares, sobre todo, teniendo en cuenta que el tiempo disponible para aprendizaje se tiende a minimizar. Por otra parte el uso de nuevas herramientas o métodos implica nuevos riesgos que solamente se descubren con el tiempo.
- **Cambio de herramientas en medio del proyecto.** Resulta muy arriesgado cambiar la herramientas de desarrollo durante el proyecto puesto que los fallos en su uso y una curva de aprendizaje más costosa de lo esperado pueden anular fácilmente lo beneficios esperados e incluso descompensarlos. El entorno tecnológico ha de ser escogido con calma antes del proyecto y ser mantenido durante su realización.
- **Ausencia de sistemas de control de versiones de código fuente.** Esto es básico, pero desgraciadamente resulta increíblemente frecuente que se destruyan partes importantes de código y recursos por la ausencia de sistemas de control de versiones y backup.

CONCLUSIONES

Desarrollar productos software es una tarea complicada. Concebir un sistema de información complejo ya es de por sí un reto, si unimos este hecho a que por lo general la tecnología empleada es muy sofisticada (y más hoy en día) queda claro que solamente se puede afrontar con éxito si se gestiona con cuidado y los medios de ingeniería adecuados. En los párrafos anteriores hemos relacionado los errores más comunes y más importantes, pero hay muchos más. Resulta muy recomendable actualizar esta lista continuamente con los errores que se detecten en cada uno de los proyectos abordados.

Resumen de los errores clásicos

Factor humano	Proceso	Producto	Tecnología
1. Motivación insuficiente	1. Planificación excesivamente optimista	1. Exceso en los requerimientos	1. El síndrome de la panacea
2. Empleados conflictivos	2. Gestión de riesgos Insuficiente	2. Plantear demasiados objetivos a la vez	2. Ahorros sobreestimados por herramientas o métodos nuevos
3. Heroísmos	3. Fallos debidos a la subcontratación	3. Requerimientos inestables	3. Cambio de herramientas en medio del proyecto
4. Añadir personal a un proyecto retrasado	4. Abandonar la planificación ante la presión del tiempo	4. Desarrollo no centrado en los objetivos del proyecto por parte de miembros del equipo de desarrollo	4. Ausencia de sistemas de control de versiones de código fuente
5. Oficinas ruidosas y faltas de espacio	5. Pérdida de tiempo en la fase preparativa de un proyecto	5. Negociación “tira y afloja	
6. Fricción entre desarrolladores y clientes	6. Cortar actividades fundamentales ante un retraso	6. Desarrollo orientado a investigación	
7. Expectativas irrealistas	7. Plantear demasiados objetivos a la vez		
8. Ausencia de apoyo efectivo de la dirección	8. Diseño inadecuado		
9. Compromiso insuficiente de los participantes en el proyecto	9. Control de calidad insuficiente		
10. Falta de “input” de usuario	10. Control de gestión insuficiente		
11. Demasiados interlocutores o interlocutores ineficaces	11. Convergencia prematura		
12. Intereses “políticos”	12. Omitir tareas necesarias de las estimaciones		
13. “Wishful thinking”	13. Planificar para recuperar retrasos dentro de la planificación inicial		
14. Confianza ciega en referencias establecidas	14. Programación “codelike-Hell”		

BIBLIOGRAFÍA

A continuación un breve listado de bibliografía interesante relacionada con los contenidos de este documento.

REFERENCIAS EN WEB

- Advanced Quality Solutions. Sección de downloads en la web, <http://www.aqs.es/downloads> - En esta sección se publican periódicamente documentos que reflejan las experiencias, opiniones y resultados de I+D de esta empresa en temas tecnología, gestión y negocio dentro del sector de las tecnologías de la información.
- <http://www.cetus-links.org> - Excelente colección de todos tipo de enlaces relacionado con la Ingeniería del software moderna. Tanto a nivel de desarrollo como a nivel de gestión, es el punto de partida perfecto para llegar a todo lo relevante de las tecnologías relacionadas con el software actuales y para estar a la última en la evolución del campo del desarrollo del software.
- American Management Association. <http://www.amanet.org/index.htm> - En este sitio se encontrarán artículos interesantes sobre todo tipo de temas relacionados con la gestión de proyectos, referencias bibliográficas sobre temas específicos, calendarios de eventos, etc.
- Eric Raymond. The Cathedral an the Bazaar. <http://www.openresources.com/documents/cathedral-bazaar/> - Un clásico de obligada lectura para cualquiera que tenga interés en la gestión de proyectos, para dos estilos opuestos de desarrollo, el modelo que el llama el “modelo de catedral”, propio de la mayoría de los desarrollos con el “modelo de bazar” propio del mundo Linux. Como figura importante en el mundo del Open Source es evidente hacia qué modelo se inclina, y lo hace con un argumento de partida contundente: si ha sido posible desarrollar un sistema operativo de la categoría de Linux con participantes en todos los lugares del mundo el modelo del bazar debe tener algo de sentido.

LIBROS & ARTÍCULOS IMPRESOS

Libros de referencia

- Ian Sommerville. Software Engineering, 6th Edition. Addison Wesley 2000.
- Kent Beck. Embrace Change. Addison Wesley 1999
- Roger S. Pressman. Software Engineering – A Practitioners Approach, 5th Edition. McGraw Hill 2000.
- Steve McConnell. Rapid Development. Microsoft Press 1996

Artículos y fuentes varias

- Barry W. Boehm. Software Engineering Economics. Englewood Cliffs, N.J.: Prentice Hall 1981
- Barry W. Boehm. Software Risk Management. Washington, D.C.: IEEE Society Press 1989.
- Bill Curtis et al. Software Psychology: The Need for an Interdisciplinary Program. Proceedings of the IEEE 1986, vol. 74, no. 8 (August): 1092-1106
- Bill Curtis. Substantiating Programmer Variability. Proceedings of the IEEE 1981, vol. 69, no. 7 (July): 846
- Capers Jones. Assesment and Control of Software Risks. Englewood Cliffs, N.J. Yourdon Press 1994.

- Carl E. Larson, Frank M.J. LaFasto. Teamwork: What Must Go Right; What Can Go Wrong. Newbury Park, Calif.: Sage. 1989.
- David N. Card. A Software Technology Evaluation Program. Information and Software Technology 1987, vol. 29, no. 6 (July/August): 291-300
- Gerald M. Weinberg. The Psychology of Computer Programming. New York: Van Nostrand Reinhold.
- H. Sackman, W.J. Erikson, E.E. Grant. Exploratory Experimental Studies Comparing Online and Offline Programming Performance. Communications of the ACM 1968, vol. 28, no. 3 (Fall): 403-424
- Harlan D. Mills. Software Productivity. Boston, Mass. 1983: Little Brown. 71-81.
- J. Richard Hackman, Greg R. Oldham. Work Redesign. Reading, Mass. 1980: Addison Wesley.
- J. Valet, F.E. McGarry. A Summary of Software Measurement Experiences in the Software Engineering Laboratory. Journal of Systems and Software 1989, 9 (2): 137-148
- James Bach. Enough About Process: What We Need Are Heroes. IEEE Software, Marzo 1996-98.
- Larry L. Constantine. Constantine on Peopleware. Englewood Cliffs, N.J.: Yourdon Press 1995
- Rob Thomsett. Project Pathology: A Study of Project Failures. American Programmer. Julio 1995, 8-16.
- Tom DeMarco, Timothy Lister. Peopleware: Productive Projects and Teams. New York: Dorset House. 1987
- Tom DeMarco, Timothy Lister. Programmer Performance and the Effects of the Workplace. Proceedings of the 8th International Conference on Software Engineering. Washington, D.C.: IEEE Computer Society Press 1985, 268-272